CS2201 Computer Organization & Architecture

L7: Addressing Modes & Instruction Encoding



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Instruction and Addressing Mode

- Instruction consists of Opcode & Operand
- Addressing modes:
  - The way in which the operand of an instruction is specified.
  - They are rules used by a CPU to calculate where an operand data value is located for a machine instruction.
  - Save instruction space and provide programming flexibility.

Addressing Modes

- The way in which the operand of an instruction is specified.
  - Implicit
  - Immediate
  - Register
  - Direct
  - Memory Indirect
  - Register Indirect
  - Indexed
  - Auto increment/Auto decrement

Implicit Addressing

- Operand is implied, No need to specify the operand
- Instruction contain opcode only
- Eg 1: CMA (Complement Accumulator)
- Eg 2: SLA (Shift Left Accumulator)

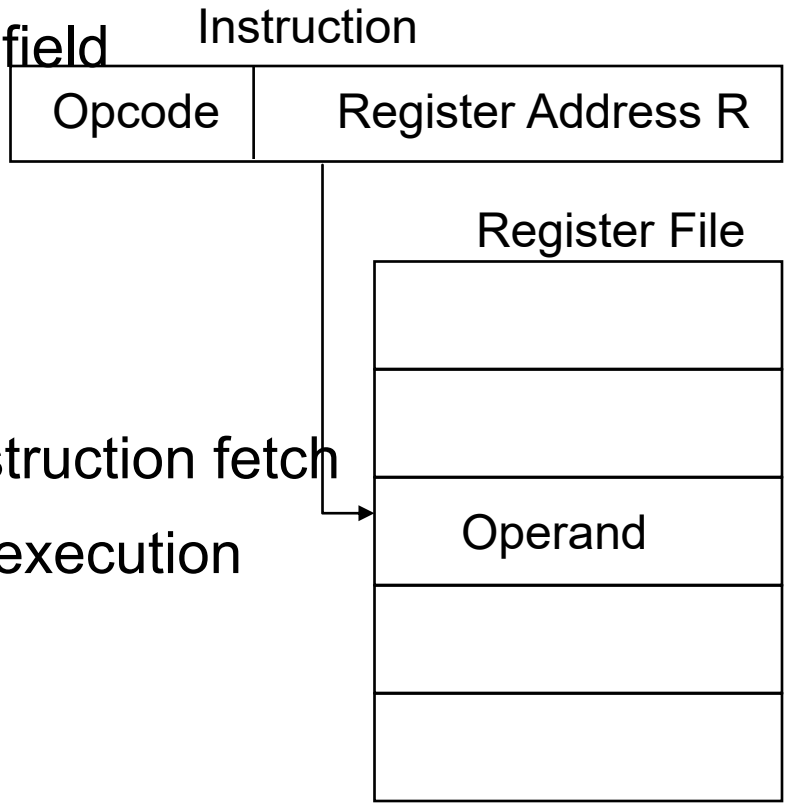
Immediate Addressing

- Operand is part of instruction
- Operand in the address field
- Eg: ADD #5
  - Add 5 to contents of Accumulator
  - 5 is the operand
- No memory reference for fetching operand. Hence it is fast
- Limited range



Register Addressing

- Operand is held in the register.
- Its name is mentioned in address field
- EA = R eg: ADD R1, R2, R3
- Limited number of registers
- Very small address field needed
  - Shorter instructions, Faster instruction fetch
  - No memory access, Very fast execution
  - Very limited address space



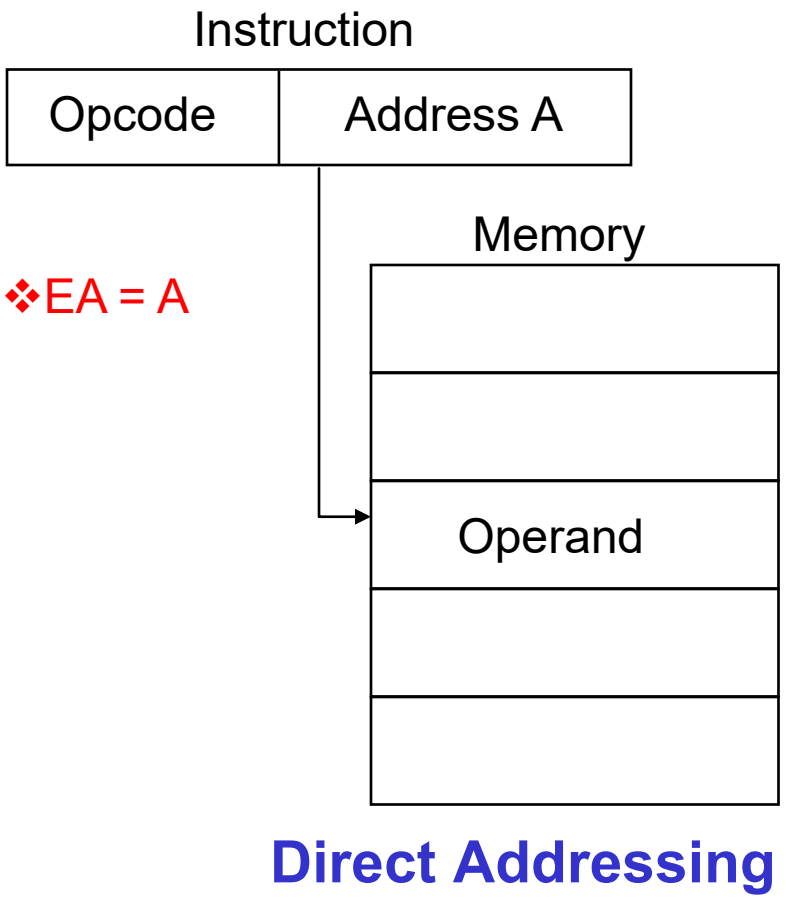
Direct Addressing

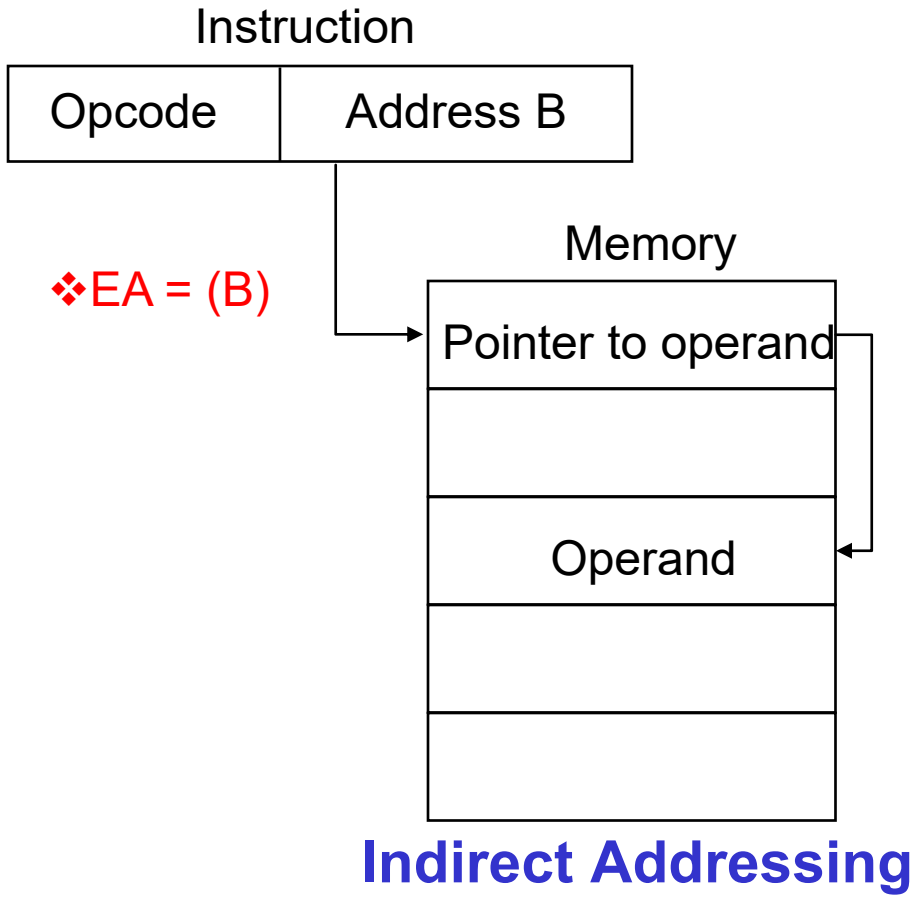
- Address field contains address of operand
- Eg: ADD 2000
  - Add contents of memory address 2000 to accumulator
  - Look in memory at address 2000 for operand
- Eg: LOAD R1, 5000
  - Load contents of memory address 5000 to R1
  - Look in memory at address 5000 for operand
- Single memory reference to access data
- Limited address space

Memory Indirect Addressing

- Memory cell pointed to by address field contains the address of the operand (indirection) → Memory Indirect
- EA = (A) → Look in A, find address (A) and look there for operand
- Eg: ADD (5000)
  - Add contents of memory pointed to by contents of 5000 to accumulator

Direct Addressing vs Indirect Addressing

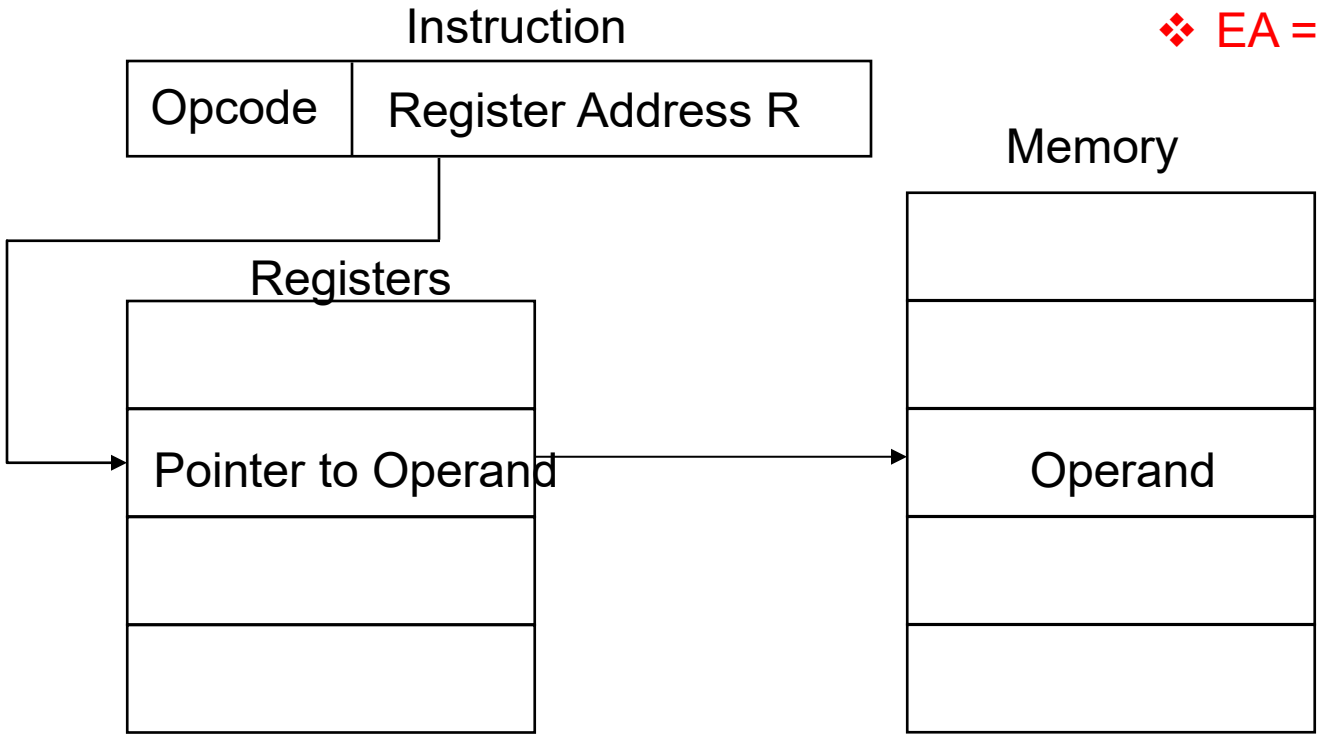




Register Indirect Addressing

- EA = (R)
- Operand is in memory cell pointed to by contents of register R
- Eg: ADD R1, (R2)
- Refer contents of R2, which is a memory address, contents of that memory location is the operand.
- Large address space (2^n)
- One fewer memory access than memory indirect addressing

Register Indirect Addressing



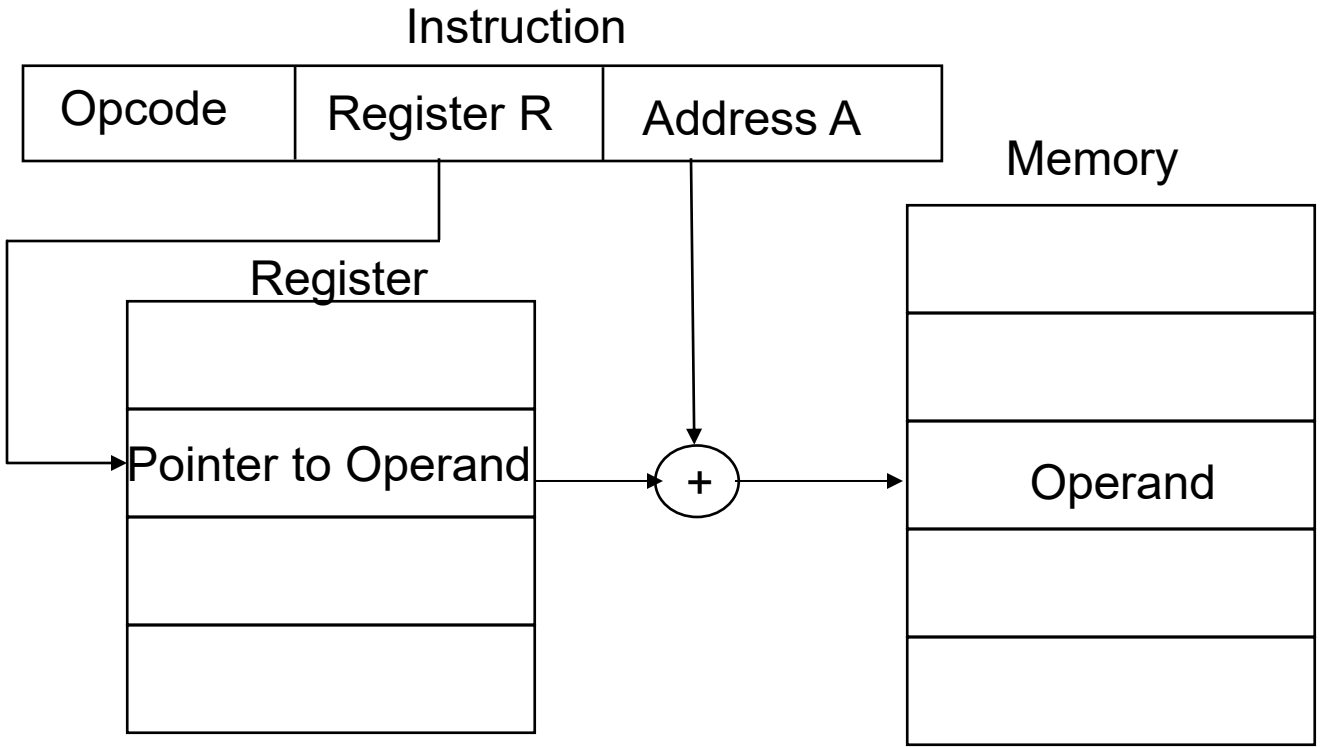
- EA = (R)

Displacement Addressing

- EA = A + (R)
- Address field is obtained by two values
  - A = base value
  - R = register that holds displacement
  - or vice versa
  - Eg: LOAD R1, 16(R2) → EA=[R2] + 16

Displacement Addressing

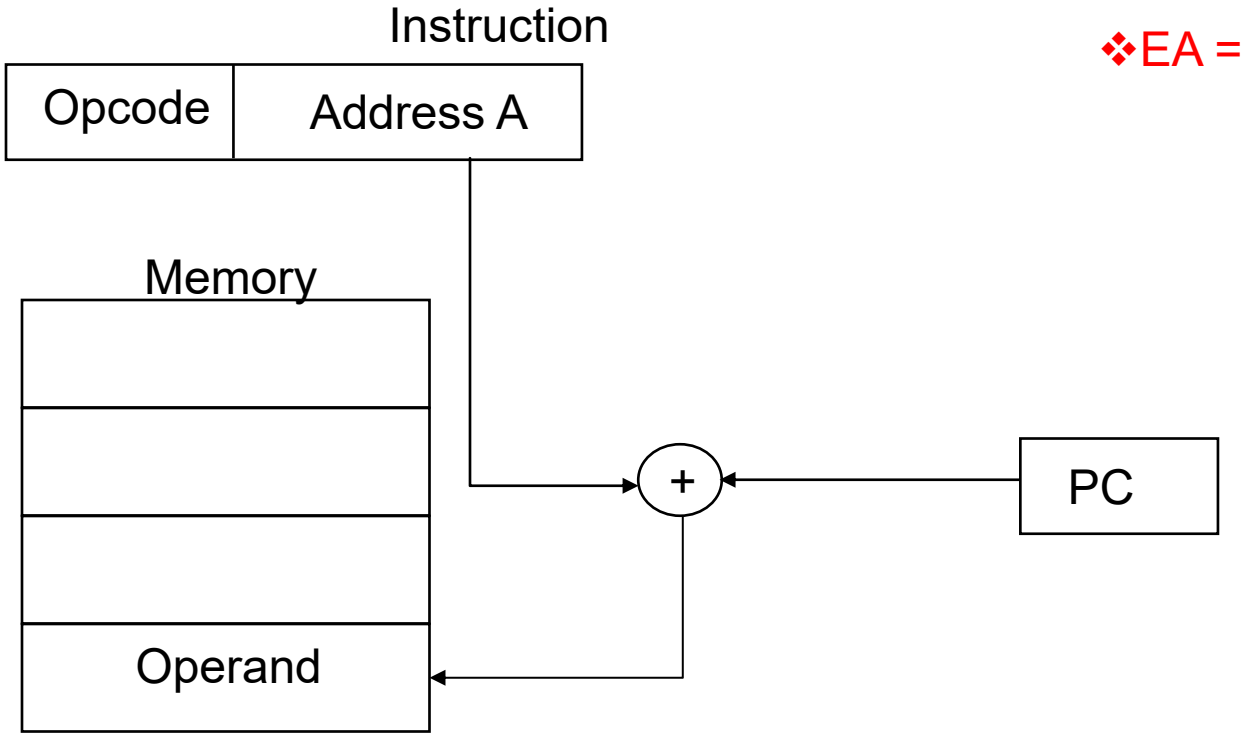
- EA = A + (R)
- LOAD R1, 16(R2)



Relative Addressing

- A version of displacement addressing
- R = Program counter, PC
- EA = A + (PC)
- Get operand from A location from current PC value [Jumps]
- Locality of reference

Relative Addressing



❖EA = A + (PC)

Base-Register & Indexed Addressing

- **Base-Register**
- A holds displacement
- R holds pointer to base address
- R may be explicit or implicit

- **Indexed Addressing**
- A = base
- R = displacement
- EA = A + R
- Good for accessing arrays
  - EA = A + R
  - R++

Auto increment and Auto decrement

- Operand is incremented or decremented after execution
- Eg 1: ADD R1, [R2]+ // R1←R1+ M[R2], R2++
- Eg 2: ADD R1, -[R2] // R2--, R1←R1+M[R2]

RISC vs CISC

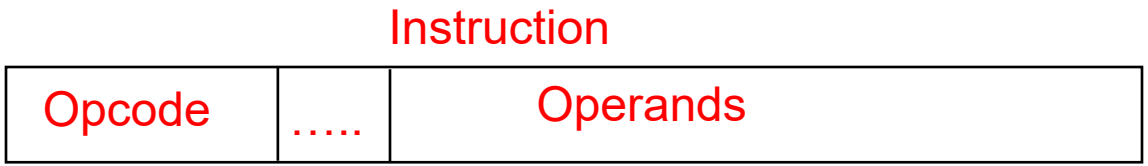
- RISC: Reduce the cycles per instruction at the cost of the number of instructions per program.
- RISC: Simple instruction, simple encoding, fewer addressing modes, more instructions for a task, large code base.
- CISC: Attempts to minimize the number of instructions per program but at the cost of an increase in the average number of cycles per instruction.
- CISC: Instructions of varying complexity, variable encoding, more addressing modes, fewer instructions for a task, small code base.

RISC vs CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

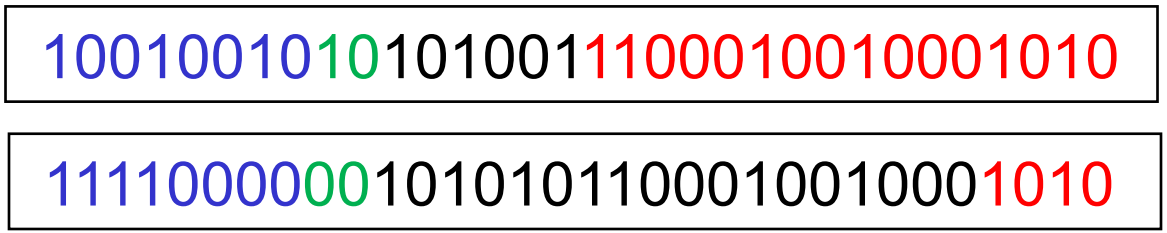
Instruction Formats

- Layout of bits in an instruction
- Includes opcode
- Includes (implicit or explicit) operand(s)
- Usually more than one instruction format in an instruction set



Allocation of Bits

- Number of addressing modes
- Number of operands
- Register versus memory
- Number of register sets
- Address range
- Address granularity



Instruction Length

- Affected by and affects:
  - Memory size
  - Memory organization
  - Bus structure
  - CPU complexity



Example – Instruction Encoding

- A processor has 16 distinct instructions (unique opcodes) and 32 general purpose registers. A 24-bit instruction word has an opcode, two registers operands and an immediate operand. How many bits are possible for the immediate operand field?
- 16 opcodes: 4 bits for opcode field
- 32 GPR : 5 bits to address them

Instruction (24 bits)

Opcode	Operand1	Operand2	Imm. Operand
4	5	5	?

Example – Instruction Encoding

Consider a 16-bit processor that uses 32-bit instruction format. It has 64 integer registers and 16 floating point registers. The instructions are broadly divided into four categories; memory instructions (M), integer register instructions (I), floating-point register instructions (F) and control instructions (C). Each memory instruction has one integer register operand, one floating point register operand and a 16-bit immediate value. Both F-type and I-type instruction have 3 operands of its respective register type and an 8-bit function code. There are 28 unique I-type instructions and 1024 unique F-type instructions. Each C-type instruction has a 24-bit offset value and an opcode field. If there are 16 unique M-type instruction opcodes, what is the maximum number of C-type opcodes possible? Illustrate a suitable instruction encoding.



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/